

Mutexes

Lightweight mutual exclusion for threads and shared resources

Definition

A **mutex** is a simplified semaphore used only for mutual exclusion. It protects a critical region by allowing only one thread or process to hold the lock at a time.

1. Why Mutexes Exist

Main Idea

When the counting ability of a semaphore is not needed, a simpler mechanism can be used. A mutex only records whether a shared resource is unlocked or locked.

Key Use

Mutexes are good for protecting shared data, shared resources, and critical sections. They are not designed to count multiple saved wakeups or multiple available resources.

State	Meaning
Unlocked	The critical region is available. A thread may acquire the mutex.
Locked	Some thread is already inside the critical region. Other threads must wait.

Representation

Only one bit is logically needed, but implementations often use an integer: 0 means unlocked, and any nonzero value means locked.

2. Basic Mutex Operations

Two Main Procedures

Mutexes are normally used with two operations:

- `mutex_lock`: acquire the mutex before entering the critical region.
- `mutex_unlock`: release the mutex after leaving the critical region.

Operation	Behavior
<code>mutex_lock</code>	If the mutex is unlocked, lock it and enter. If it is locked, block or yield until it becomes available.
<code>mutex_unlock</code>	Unlock the mutex and allow one waiting thread to acquire it.
<code>mutex_trylock</code>	Try to acquire the mutex, but return failure instead of blocking if it is already locked.

Cooperation Required

Mutexes are not magic. All threads must follow the protocol: lock before touching shared data and unlock after finishing.

3. User-Space Mutex Implementation

Why User-Space Mutexes Are Useful

Mutexes are simple enough to implement in user-space thread packages, especially when the hardware provides an atomic instruction such as TSL or XCHG.

```
mutex_lock:
    TSL REGISTER, MUTEX ; copy mutex to register and set mutex to 1
    CMP REGISTER, #0 ; was mutex zero?
    JZE ok ; if zero, mutex was unlocked
    CALL thread_yield ; mutex is busy; run another thread
    JMP mutex_lock ; try again

ok:
    RET ; critical region entered

mutex_unlock:
    MOVE MUTEX, #0 ; unlock the mutex
    RET
```

Crucial Difference from Spin Locks

If this mutex cannot acquire the lock, it calls `thread_yield` instead of spinning forever. That gives another thread a chance to run and release the lock.

4. Why Yielding Matters for User-Level Threads

Problem with Busy Waiting

In a user-level thread package, there may be no clock interrupt that preempts one user thread and schedules another. If a thread busy-waits for a lock, it may run forever and prevent the lock holder from running.

Role of `thread_yield`

`thread_yield` calls the user-space thread scheduler. The waiting thread voluntarily gives up the CPU, allowing another thread to run.

Effect

If thread A tries to lock a mutex held by thread B, A yields. B can then run, leave the critical region, and unlock the mutex. Later A can try again and succeed.

Performance Benefit

Because `thread_yield` is a user-space scheduler call, `mutex_lock` and `mutex_unlock` can avoid kernel calls in the uncontended user-thread case.

5. Trylock

Trylock

`mutex_trylock` attempts to acquire a mutex. If the mutex is available, it locks it. If the mutex is already locked, it returns an error code instead of blocking.

Why It Is Useful

Trylock lets a thread choose another action instead of waiting. For example, it may do different work, retry later, or avoid a possible deadlock pattern.

6. Shared Address Space Issue

Threads

Threads in the same process naturally share an address space, so they can all access the same mutex variable.

Processes

Separate processes usually have separate address spaces. For them to share a mutex, semaphore, buffer, or Peterson-style variable, some shared storage must exist.

Approach	Meaning
Kernel object	Semaphores or locks can be stored inside the kernel and accessed through system calls.
Shared memory	Modern systems can map part of one process's address space into another process.
Shared file	If nothing else is available, processes may coordinate through a shared file.

Processes vs. Threads

Even if two processes share much of their address space, they are still not the same as threads. Processes still have separate per-process properties such as open files and timers, while threads in one process share them.

7. Futexes

Futex

A **futex**, or **fast user space mutex**, is a Linux synchronization mechanism that tries to avoid kernel calls when there is no contention, but uses the kernel to block threads when waiting is necessary.

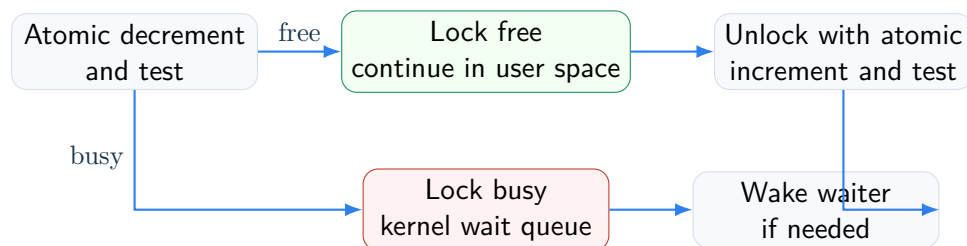
Motivation

Spin locks are fast when waits are very short, but waste CPU cycles under heavy contention. Blocking in the kernel is better under contention, but expensive when locks are usually free.

Best of Both Worlds

A futex stays in user space when the lock is uncontended. It enters the kernel only when a thread must actually wait.

8. How a Futex Works



Typical Futex Lock

Processes share an aligned 32-bit integer as the lock. If an atomic operation shows the lock was free, the thread acquires it in user space. If another thread already holds it, the library makes a system call to put the thread on a kernel wait queue.

Typical Futex Unlock

When the holder releases the lock, it uses an atomic operation and checks whether any threads are blocked in the kernel. If so, it asks the kernel to wake one or more waiters.

9. Pthreads Mutexes

Pthreads Mutex

In Pthreads, a mutex variable is used to guard a critical region. A thread locks the mutex before entering and unlocks it after leaving.

Thread Call	Description
<code>pthread_mutex_init</code>	Create or initialize a mutex.
<code>pthread_mutex_destroy</code>	Destroy an existing mutex.
<code>pthread_mutex_lock</code>	Acquire a lock or block if it is already locked.
<code>pthread_mutex_trylock</code>	Acquire a lock or return failure without blocking.
<code>pthread_mutex_unlock</code>	Release a lock and allow one waiting thread to continue.

Not Mandatory

Pthreads mutexes do not automatically protect data. The programmer must consistently use the correct mutex around every access to the shared data.

10. Condition Variables

Condition Variable

A **condition variable** lets a thread block until some condition becomes true. It is usually used together with a mutex.

Why Mutexes Alone Are Not Enough

A mutex can protect the check of a condition, such as whether a buffer is full. But if the condition is not satisfied, the thread needs a way to block and be awakened later. That is the job of a condition variable.

Thread Call	Description
<code>pthread_cond_init</code>	Create or initialize a condition variable.
<code>pthread_cond_destroy</code>	Destroy a condition variable.
<code>pthread_cond_wait</code>	Block until another thread signals the condition variable.
<code>pthread_cond_signal</code>	Wake one waiting thread.
<code>pthread_cond_broadcast</code>	Wake all waiting threads on the condition variable.

Atomic Wait

`pthread_cond_wait` atomically unlocks the mutex and blocks the calling thread. When the thread wakes up, it reacquires the mutex before returning.

No Memory

Condition variables do not remember signals. If `pthread_cond_signal` is called when no thread is waiting, the signal is lost.

11. Mutexes and Condition Variables Together

Producer-Consumer Pattern

For a buffer, the mutex protects the shared state such as `count`. Condition variables let the producer wait while the buffer is full and let the consumer wait while the buffer is empty.

```
pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

int count = 0;
int buffer;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();

        pthread_mutex_lock(&mutex);
        while (count == 1)
            pthread_cond_wait(&not_full, &mutex);

        buffer = item;
        count = 1;
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        pthread_mutex_lock(&mutex);
        while (count == 0)
            pthread_cond_wait(&not_empty, &mutex);

        item = buffer;
        count = 0;
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);

        consume_item(item);
    }
}
```

Always Recheck the Condition

The waiting thread should use a `while` loop, not a simple `if`. A thread can wake up even though the condition is still false, so it must check again before continuing.

12. Mutexes vs. Semaphores vs. Condition Variables

Primitive	Main Job	Key Detail
Mutex	Mutual exclusion	Locked or unlocked; does not count resources.
Semaphore	Counting and synchronization	Stores wakeups and can count available resources.
Condition variable	Waiting for a condition	Has no memory; must be used with a mutex and condition check.

13. Quick Review

Key Points

- A mutex has two logical states: unlocked and locked.
- Mutexes are simplified semaphores used only for mutual exclusion.
- `mutex_lock` acquires the lock or waits.
- `mutex_unlock` releases the lock and may wake one waiter.
- User-space mutexes can use TSL or XCHG plus `thread_yield`.
- `mutex_trylock` fails instead of blocking when the mutex is busy.
- Processes need shared storage or kernel objects to share synchronization variables.
- Futexes stay in user space unless contention requires kernel blocking.
- Pthreads provides mutex calls and condition-variable calls.
- Condition variables have no memory, so waiting code must recheck the condition.